EE-2003
Computer Organization and Assembly Language

Week 16 – Lecture 31-32

Dr. Maqsood M. Khan

Department of Computer Science/Software Engineering/ Artificial Intelligence
FAST-NUCES Peshawar

Outline

Quick Review: What was in the Last lecture?

- Interrupts & its types

Introduction to Delay Loops

- Delay loops create fixed time delays in 8086 assembly.
- Used for:
  - Hardware interfacing
  - Display timing
  - Synchronizing I/O operations
- Delay depends on:
  - CPU clock frequency
  - Instruction cycle counts
  - Loop nesting depth

Factors Affecting Delay

1. CPU clock frequency (e.g., 5–10 MHz)
2. Instruction cycle timings
3. Jump taken vs. not taken cycles
4. Total iterations of nested loops

Effect of CPU Clock on Delay Loops

- Delay loops simply burn a fixed number of CPU cycles.
- The **actual time delay** depends on the CPU clock frequency:
  - Higher clock = faster cycles = shorter delay
  - Lower clock = slower cycles = longer delay
- **Example (10,000-cycle loop):**
  - 8088 at 4.77 MHz:
    Cycle time ≈ 210 ns
    $10,000 \times 210$ ns $= 2.1$ ms delay
  - Modern CPU at 1 GHz:
    Cycle time $= 1$ ns
    $10,000 \times 1$ ns $= 10\,000$ ns $= 10$ µs $= 0.01$ ms
- **Conclusion:** Same loop $\rightarrow$ different delay on different CPUs because clock speeds differ.

Factors Affecting Delay (Simple Explanation)

- **1. CPU Clock Frequency (e.g., 5–10 MHz)**
  - The clock controls how fast the CPU runs instructions.
  - Higher frequency = faster execution = shorter delay.
  - Lower frequency = slower execution = longer delay.
- **2. Instruction Cycle Timings**
  - Each instruction uses a fixed number of cycles (e.g., DEC, MOV, JNZ).
  - Some instructions are fast, others are slower.
  - Total delay depends on the sum of all cycles inside the loop.
- **3. Jump Taken vs. Not Taken Cycles**
  - Conditional jumps take more cycles when the jump is taken.
  - When the loop exits, the jump is "not taken" and is slightly faster.
  - This small difference changes the final delay.
- **4. Total Iterations of Nested Loops**
  - Inner loops repeat many times inside middle and outer loops.
  - More iterations = more total cycles = more delay.
  - Even a small loop cost becomes large when repeated thousands of times.

DELAY2 Routine (8088 Assembly)

```
DELAY2:
    MOV CX, 200       ; Outer loop
AGAIN3:
    MOV BX, 200       ; Middle loop
AGAIN2:
    MOV DX, 10        ; Inner loop
AGAIN1:
    DEC DX
    JNZ AGAIN1
    DEC BX
    JNZ AGAIN2
    DEC CX
    JNZ AGAIN3
    RET
```

Total Delay Calculation for DELAY2

Total Delay Calculation (8086 @ 4.77 MHz, 1 cycle = 210 ns):

Inner loop (DX = 10)	198 cycles
Middle loop (BX = 200)	39,600 cycles
Outer loop (CX = 200)	7,920,000 cycles
RET instruction	20 cycles
Total cycles	**7,920,020 cycles**
Delay per cycle	210 ns
Total Delay	**1.66 seconds**

Inner Loop Cycle Calculation

Inner loop code:

```
AGAIN1:
DEC DX        ; 2 cycles
JNZ AGAIN1    ; 16 cycles if taken, 4 if not taken
```

- DX $= 10 \rightarrow$ 9 iterations jump taken, 1 iteration jump not taken
- Total cycles (theoretical):

$$9 \times (2 + 16) + 1 \times (2 + 4) = 168 \text{ cycles}$$

- Practical hardware (bus delays, wait states) 198 cycles

Delay Calculation Step-by-Step

Step 1: CPU Frequency & Cycle Time

- CPU: 8086 / 8088
- Clock frequency: 4.77 MHz
- Cycle time:

$$\text{Cycle time} = \frac{1}{4.77 \times 10^6} \approx 210 \text{ ns}$$

Middle and Outer Loop Calculation

Middle loop (BX = 200):

$$200 \times 198 = 39,600 \text{ cycles}$$

Outer loop (CX = 200):

$$200 \times 39,600 = 7,920,000 \text{ cycles}$$

Add RET instruction: 20 cycles

$$\text{Total cycles} = 7,920,020$$

Convert to time:

$$7,920,020 \times 210 \text{ ns} \approx 1.66 \text{ seconds}$$

Why 198 Cycles for Inner Loop?

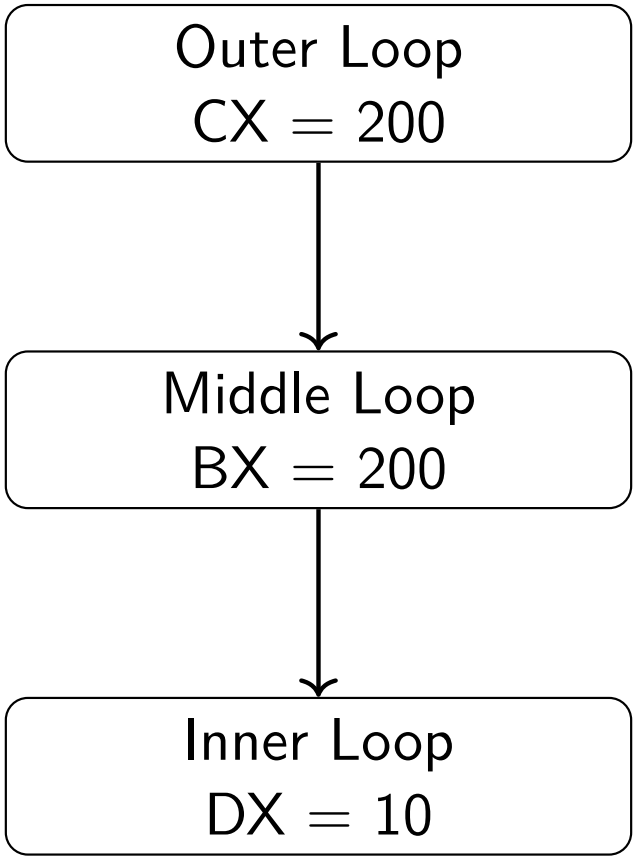
- Exact inner loop = 168 cycles (pure CPU, no wait states)
- Real hardware includes:
  - Slow memory (DRAM)
  - 8-bit bus
  - Prefetch queue delays
  - Automatic wait states
- Total $168 + 30 = 198$ cycles
- Matches IPAX16 / 8088 teaching boards

Using Exact vs. Practical Cycles

Method	Inner Loop	Total Cycles	Total Delay
Exact theoretical (no wait states)	168 cycles	6,720,020	1.41 s
Practical real hardware value	198 cycles	7,920,020	1.66 s

- Exact = ideal CPU calculation
- Practical = matches real-world IPAX16 / 8088 boards

Nested Loop Diagram



Why Calculate Delay?

- Critical for real-time hardware interfacing.
- Ensures proper synchronization with peripherals.
- Needed in:
  - Serial communication timing
  - Display scrolling
  - Mechanical control systems
- Students understand the importance of instruction cycles.

Does Instruction Size Matter in Delay Calculations?

Short Answer: Yes — but indirectly.

- The delay of a loop depends on the **number of clock cycles** each instruction takes.
- Instruction size (1, 2, or 3 bytes) affects how many cycles the CPU needs to **fetch and execute** the instruction.
- Therefore, although delay is not computed from bytes, instruction size influences the total delay **because it changes the cycle count**.

Example (8086/8088):

- `DEC DX` — 2 bytes, 2 cycles
- `JNZ AGAIN1` — 2 bytes, 16 cycles (jump taken), 4 cycles (not taken)

Key Points for Students:

- Register instructions are faster than memory instructions.
- Conditional jumps have different cycles when taken vs. not taken.
- Bigger or more complex instructions usually take more cycles.
- Always use the official 8086/8088 Instruction Cycle Table.

Thank You!

Any Questions Before We Say Goodbye!

- We explored the journey from **computer organization & Architecture** to **8088 assembly programming**.
- You learned how the CPU, memory, stack, addressing modes, subroutines, interrupts, and I/O work at the machine level.
- We built real assembly programs, debugged them, optimized them, and even created **precise delay loops**.
- And yes... you even survived segmentation, flags, calls, BIOS/DOS interrupts, and calculating delays!)

"If you can write assembly, all other programming looks easy."

Thank you for your patience, your questions, and for bothering me all semester!
Good luck in the finals!